ActInf MathStream 012.1 ~ ELBOing Stein: Variational Bayes with Stein Mixture Inference, Ola Rønning

Source: YouTube **Playlist**: MathStreams **Duration**: 58:17 | **Uploaded**: Unknown
Transcript method: auto_caption

Hello, welcome. It is April 10th, 2025. We're in active inference math stream 12.1 with Ola running. Going to be discussing elbowing stein variational bays with stein mixture inference. So, thank you for joining. Looking forward to this presentation and discussion. Great. Thanks for having me. Um yeah, I'm all running and presenting the work I've done together with Distri Smith and Thomas Hammer on varational base uh with Stein mixtures. Um so let's get into it. So uh this is the plan. We'll talk about inference and particle based methods um and particular uh steinbased methods which came about in 2016. just go over uh an overview of what these are to catch everybody up and then the key problem with them which is the underestimate variance as our models get high dimensional uh then I'll show shortly uh shortly about our approach which is d mixture inference and the new thing here is introducing um a neighborhood to each of the particles we'll talk a little bit about that and then evidence that this mixture actually helps with the issue of dimensionality curse of dimensionality. And finally, Stein mixture in French is a um is a blackbox inference engine in the proistic programming language on pyro. So I'll plug that a little bit at the end. Yeah. So um just to set the stage here. So making predictions and sort of modeling uh systems is ubiquitous. Uh so the group I'm in works with uh um structural biology. Uh but I'm looking to applications in in robotics and potentially finance as well. And and sort of at an abstract level, what we're looking for is a trustworthy systems. We want something that's accurate when it meets ex when our data meets our expectation and uncertain when it's either surprising, ambiguous, or missing. So illustrating it here by where we have a we have a data generating process dotted line and then we have a method which is trying to infer um uh to infer the process. This is the red line and so we only we only give it uh sufficient data in the green regions and what we're looking for is something that captures the uh the the the process in the green region and gets uncertain in this in between region. And so on the left hand side you would have something that's accurate but

it's uh but it's overly confident um because it the uncertainties uh is collapsed in the in between region and the right hand side is what we're uh what we're what we want. So here the blue region could be any potential curve um and it's like 95 or 90% HDI. So it's like the likeliest would be in here. Good. Um, yeah. And so just to make it slightly more concrete, so in prediction, you might have something like a overconfident uh overconfident mobile uh mobile robot. So if you for example have a uh robot navigating no terrain, um it will have a sort of uncertainty about its orientation and uh uh and position. uh but as you put it into new terrain for example by removing a tree you would want this uncertainty uh to to spread out so that you can invoke emergency protocols and an over confident protocol would uh uh sorry overconfident a robot might lead to crashes because it's there's a disambiguity between its actual location and where it believes it's at. And this is a this is a we have a talking with some underwater uh robotics labs and they say that this is an issue in one out of€ 250 missions and this can be quite expensive with robot costing between 20,000 and 10 million depending on on the size of the robot for deep sea robots. another issue. So that was sort of when we're talking about prediction, but when we're talking about modeling, uh uh protein structures, uh threedimensional protein structures have uh regions which are sort of conserved, so they stay um um or ordered, so they don't fluctuate very much, but it's it's a it's a molecule interacting with the solvent around it. uh so the whole thing will move slightly and then you have some regions which are unstructured and highly disordered and they will be moving around. So if you want a uh a description of the structure it's not enough uh of the three-dimensional structure of the protein. It's not enough to know just um um sort of an image of of the structure cuz there's might be some areas which are not which will only be very or very unlikely to be in that particular position. So you want some sort of a representation which de uh captures that you have flexibility within the mod u molecule and uncertainty is a natural way of representing this. Okay. So and and the way we are going to go about this is with bashian inference. So just a short reminder uh of bashian inference. So we're given observations uh x. So this is things we get to see and we have a model with some uh latent parameters unseen parameters which I'm going to call theta throughout. And then we can apply base rule to get to uh take our beliefs uh prior beliefs about theta and incorporate uh the observations we're seeing to get to our posterior. Uh and we commonly use this posterior to either we want to sample from it which is the techniques I'm going to show you here uh or on some cases you want to compute expectations with respect to the posterior. Now we have two challenges. This first challenge is key like is important in terms of what we're doing here which is our model of

data uh is highdimensional and the other problem uh which something you work around when you're doing this type of uh algorithmics is you don't want to evaluate the nom length in constant because it's uh interactable analytically interactable at least so uh so so this methodology sort of first came out uh came about in 2012 and the Key idea is to sort of take a a simple me uh measure and then find a um a a push forward sort of deterministic map such that you transform this very simple measure into the target measure and our target measure in in patients inference setting is is our posterior and uh so this is one aspect of it we're trying to find this deterministic map and uh secondly we're going to move not the entire density because that would require us to compute the normal normalizing constants. We're only going to be comput uh we're only going to be moving particles. So we're going to move something that looks like samples from this simple distribution towards target distribution. And so the uh multi and masuk did this using uh what was it uh orthogonal polomials. So like they had this basis of polinomials and then they found so of one constructed one function which did the move the entire transport. Uh that was in 2012 and then in 2016 uh Steinverian grad descent came out. So the way they find that they determine the transport is they uh they choose a distance between distribution. So the simple distribution row and the target distribution the posterior um and then you apply your transport and sort of they uh Kyouong and Dilling did two things. The first thing was they deconstructed the transport into an iterative process um and secondly they sort of gave a simple form to each of these transports which is just a pertubation to um to our identity map. Um and then after iterating this process, they would then make sure that each iteration you would get closer to uh uh from the from the from this sort of simple prior metric towards the stereo metric and and that the key result here is that they get a close form for the sort of finding this vector field which is making taking uh giving us the direction of our transport. So just taking this apart uh the name apart we see the stevational gradient descent. So it's a gradient descent like uh algorithm we get a update rule which looks like the classical gradient descent only our our thetas here are random variables and not the parameters as we would be used to and the way it works is we draw a set of particles from uh from our prior. this set of particles we put an empirical measure on um and then we compute or we find the minimizing distance uh in terms of scale uh towards for the next step and we apply that transport so that uh that that vector field to the particles rather than the entire density and this eventually converges uh to our target target density and then there's the stein part so this is going to get us uh uh get us the direction in which to move particles. Um so uh what they find is they find a direction which is the maximal decreasing uh direction

in the K in terms of KL uh where the vector field is contained within a large enough class of functions called big okay and so uh this was known before but there's a relation between uh the kale divergence when you have this particular form uh of transport and uh the stein uh well kernstein discrepancy and so this kernstein discrepancy is really looking at an operator operating on the vector field and it takes this particular form turns out there are more but this is the most common but common form and here um uh we have the score function and then we have we have our vector field I'm just because I had questions about this before we oh this is an old version sorry uh the score function allows us to this should be theta down here and then we don't get to evaluate the we don't have to evaluate the um the gradient with respect to the evidence or sorry the derivative with respect to to the evidence that will just be zero. So we can evaluate the score by just having a joint uh over theta and x over a lat observations. And the second insight uh from that one was sort of well we can have a a form for the for the for the vector field but we can actually also compute it uh if we choose the space correctly. Uh oh that should be uh and and what the space of functions that we're choosing is reproducing kernel of hbert space uh and where where our what we get is with a kernel is the partial evaluation uh is a point in the hbert space. So if we have a way of uh if we have uh have kernels available to us with simple functional forms then we also have a way of computing uh computing the optimal direction um and and Colonel Hilbert spaces or sorry uh reproducing colonel Hbert spaces have sort of been studied both in uh in u in gshian processes and uh and I think support vector machines before this. we have quite a few available. Um the one we're going to be focusing on here is the squared exponential kernel. Uh functional form is given here. Uh there's quite a few different ones. There's also so these are scalar kernels and what you do is you then take identity uh matrix to make it to into a a vector field over uh over RD. There's also matrix variants and then we have operators which allow us to combine kernels and and there's a lot of equations here and I sort of this is a simpler view of the whole thing. So uh if we we have a kernel and it's it's partial evaluation gives us a kernel in the function hbert uh in reproducing kernel hbert space. So for example a constant kernel um that just our constant will decide uh exactly which value we take. Um in a linear kernel you would have something like um the kernel sorry the the the partial evaluation whichever one we choose will choose the the the slope of our function and the key one we'll be working with is the squared exponential also known as the gshian and there the partial evaluations uh give us the mean whereas we have a a hyperparameter we have to set ourself which is the bandwidth and that is going to choose sort of the the width of our of our caution. C can you just clarify what the red

and the blue are are for these different kernels? Yeah. So these are I'm taking a kernel, right? A kernel is a function in this case it's just R goes to R goes to R. So X here is a free variable and and and and our number is a fixed one of the parameters. And so uh the blue dotted line corresponds to fixing one of the parameters to two. The gray one is fixing it to one sorry to zero and the red is fixing it to to 2 minus 2. Is there a reason for two and negative -2 or is this just showing like the two directionalities you can take from zero or is there something about the two? No, there's nothing particular about choosing min. It's choosing both sides of uh when we just have when we just have a line, right? Uh it was just to show illustrate sort of what happens with the only interesting boundary here which is over zero. Awesome. So they're like the two directions you can go updating the kernel. Yes. if we're in one dimensional space and then they all have hyperparameters like a length scale or uh which I've sort of shown down here but they also just chosen arbitrarily just to illustrate uh this functional view. All right. So uh recall that I showed you this uh this functional form of the update just writing it out uh with expectations. So right now I'm just applying it to two two particle system we get a we get something that uh that behaves like we see this vector field actually behaves like forces acting on physical particles. So the first term is a defractive uh force. It's moving towards the nearest mode in our posterior and it's moving each of the particles like we're taking the it's a mean field method right so we're taking the average uh effect uh and moving in the particle according to that so we have the effective force in blue and then we have a repulsive force which acts pair-wise between particles and that is keeping the particles apart uh so they will have uh so they will not collapse onto uh each other if we only had the first uh the first term here, the attractive force, we would have no benefit of having more than one particle uh because everything would just collapse to the news and then put in our final system. So we initialize like these positions randomly. We run this system forward and what we get is something that looks a akin to samples from the posterior. They are not exactly samples which is also why we call them particles. they have they have correlation structures which are sort of different than you would have MCMC but they do certainly retain some sort of correlation structure I don't think it's well characterized though at the moment so uh so that sort of gives an idea about a particle approach and so uh particle approaches like in one dimension these they're shown like they are as good good as MCMC but as we go up in dimensionality we start seeing issues. So yeah, I'm doing the same system I was doing before. I'm using a basian neural network which has in the low dimensions 41 dimensions and the high 10,100 uh 4001 uh dimension and we we capture some uncertainty uh in this in between region when we're

using in the load uh uh in the low uh dimensional system but as we move up in dimensions this completely collapses. Now the weather basian network sort of how many parameters also decides um um how expressive our model is and so uh sort of as you can see with the ideal you get less expressive like you there's fewer curves you're actually able to fit uh with a lower dimensional BN which is why you see a difference in terms of uncertainty in the ideal case going from low to moderate So interesting question then is well when do we sort of stop trusting stein version of gra descent and uh Jimmy B in 2001 came up with an interesting experiment where you have um Jian we let it dimen multivariate gshian let its dimensionality grow slowly and then you're looking at so what is the average uh empirical variance you get per dimension of your estimate. And what you see with dimensional grain descent is already at five dimensions. You've sort of have this uh marginal variance. Uh and so you should really not trust uh uncertainty estimations if you have a problem which is five or higher dimensional and there's a lot of these. How does that relate to the factorization? Like the difference between doing five parameters in a joint distribution or factorizing some of the variables apart. I'm not sure I completely follow like when you're talking about the dimensionality increasing this is doing a joint distribution of all of those okay in their possible mutual influence. Yeah. So here we're actually so our um our multivariate gshian is assumed completely independent which is why we can compute this very simply easily. So it's interesting question whether how it actually works if you start having coariances as well but this experiment is really the simplest version you could imagine where you just have a okay and and if there were covariances do you think that would make the situation worse or more tractable? I would assume I I would assume it would make it worse. Uh this is sort of the simplest version of the system I could imagine. Uh so this would make it worse. So this is sort of a lower bound on what we're uh what we should expect. Um yeah. Yes. Good. So onto our approach sign inference. So what we uh propose is to have separate the particle space and the uh the laten parameter space so that you get a hierarchical model and you let the uh the uh the particles move in this uh in the site uh space and then you let each particle parameterize uh its own variational distribution which then exists in the same space as our posterior. And so what you get is you get a mixture approximation of the same uh operational distribution uh to our posterior and uh here m is the number of particles. So it would be two. So if you're using 10 particles then you're using a then you would have a a 10 component mixture approximation and just to motivate it uh we can rethink sort of we can rethink the the uh steinber should be descent particles as sort of the take if you could imagine taking up gshian and then letting the

variance go to zero you would get something that's just be a direct delta function uh distribution but it would just be like a a a probabilistic atom. And so you can think of in this sense you could think of sort of a stein particle as just the mu location of this sort of degenerate caution. And so going in the opposite direction, you could say well uh if we then uh allow the variance uh of of the of the normal to also be represented by the particle then each particle now becomes two-dimensional. um and and and and summarizes this caution. So so we this our particles now both have a location but they also have a component for the variance. Um and then with multiple uh like having many of these particles you would suddenly get a mixture approximation and in this case a gshian mixture approximation. And so our goal now is to be able to move these particles uh in the in in um in such a way that we're sort of minimizing a distance to the uh to the posterior to get a uh version of base method. And to do this we we're going to look back to sort of meanfield version of inference and just go here. You have a you have a simple parametric distribution. So we not we're not necessarily trying to hit fix uh hit uh get the shape right of our posterior. However, we are going to work with something that's of uh that is tractable. And so uh here you just a gshion summarized by this this midline its mean and we sort of then move the mean such that it's covering as well as it can uh the posterior distribution and we'll do this with a proxy distance. Um so it will not be the kale we're minimizing anymore. It will be evidence lower bound. And so just to recall that as well, it's it's our evidence lower bound is just the difference between the our evidence of yeah log probability of our data and the K between our approximate uh distribution Q and our true posterior and it's and it's this expression and the reason we work with this is this this expression here which compute especially if we have a very simple rational distribution Q we might even be able compute the the differential entropy analytically. Good. So the idea of uh so as I mentioned we were going to take uh our particles and give them a neighborhood. So in terms of of uh of the equation I was showing you before, we could think of making these uh the parameters uh of our rational distribution, we could make those into random variables. And if we do that then uh we would get a and we had multiple of them then we have a uh we would have an empirical measure on uh on the parameters of our of our rational distributions and it would be the same as replacing the score function the stein version of gradient descent uh with with some sort of uh of loss function here which is going to be similar to an elbow. Now we'll have to do a bit of massaging to make sure that this is still a valid uh meth methodology. But sort of in its sort of simplest form, this is all we're doing. We're we are moving taking our transport. We are moving on these particles which are now a hierarchy higher than it was

before and we are moving them using uh using this uh this elbow formulation instead uh of of in terms of the of the score function and the m uh the evidence lower bound that we're using is takes this particular form. You could use simpler versions of this but it actually it behaves doesn't behave well. uh so we need to take in particular we need to take this uh the differential entropy in terms of uh of our rational approximation not just each of the terms. Um in order to show that this is correct we do have a constraint on this which is our guide distribution operational uh uh distribution Q that needs to be the same uh for every single particle. In other words, our our our function our function of row uh needs to be symmetric. So it can't depend on the ordering in which you uh in which you uh parameterize it. With this change uh we can use uh an extension of time gradient descent called nonlinear stevial gradient descent. And what this is again dealing on le what they did was they replaced so it's dime version of B descent we're minimizing the KL divergence uh and what this extension does it allows us to minimize um functionals of our initial distribution row uh with our uh differential entropy as our as a regularizer and what that is doing is in of ensuring uniformity so ensuring that our our density row gets spread out there's some prerequisites for this. First of all, as I already mentioned, uh our functional when uh parameterized by uh by empirical measure needs to be symmetric. Secondly, we need to have a differential evaluation map. Oh, this is again the old version. I'm sorry. This should be this is all the way up to M. There's nothing special about M here. Um but we need to be able to to to evaluate uh the functional given our particles and using an empirical measure that's trivial. It's just evaluating at each of the each of the points and finally we have this regularizing term which is ensuring that our particles are spread out. So our uh as we show okay but this is uh we have an instance of nonlinear stress gradient descent uh first of all it fulfills that the particle approximation is symmetric because we chose the same que for each of our each of our articles so the same uh guide we have a differentiable map but it's trivial to evaluate on the empirical measure and if we choose it needs to be differentiable so we also choose the uh our guide to be differentiable and finally Finally uh we were when I showed you we were replacing sort of the inner part of the of the vector field by EV evidence lower bound but recall we're now this is the objective we're actually optimizing is the this functional uh plus some entry picture in Monroe which is up here and so we need to show that that is also an evidence lower bound in order to have a valid version uh approach and if you choose so they have this alpha parameter if reduce that to one and you look at the population limit. So like infinitely many particles and I'm moving this um then we actually we can actually show that this is the case. So it's a valid fractional base uh method. And so if you

revisit the experiment I was showing you in the beginning um with uh within version gradient descent we had collapse. And now with our stein mixtures we see that we're going up in dimensionality our particles uh are in in theta space. So in the some posterior space uh our particle we now have uncertaintity estimation in this uh in the moderate dimensional case as well. However, it's not uh it's not ideal yet. Uh we are here we're using five particles and so you can get better approximations using more but the cost of the sort of the cost of each step is uh quadratic in the number of particles. So you want to use as few particles as possible. Um but but there's so there's there's still room to uh for improvement. Uh but but we are sort of work it works better than than uh than steep for this uh uncertainty estimation. So as I said we we we sort of there's still improvements to be made. There's some key design dimensions that we can address. crystal dynamic stability which is we sort of stress gradient descent has been understood as sort of gradient flow in uh in in the space of distributions. However, if you just tune that naively with mixture models then you might move your gradient flow might move somewhere which is no longer a mixture model. So we in order to sort of analyze and understand uh uh sty mixture inference you need some some new theory there. Uh but with this if you get this gradient uh flow uh if you nail that then you start being able to do things like you can you can choose hyperparameters by choosing different formulations of the gradient flow. uh and and and for example look at uh time flow versus uh this is getting to technology sorry uh um what you can do you can start uh determining something like kernels based on having uh this type of formulation it also allows us to tell things about how fast it converges which is currently not known uh the result on stress gra descent at IC law this year which shows that it's in KSD it's actually as good as uh as uh MCMC. So that's very good in terms of KST. Then there's kernel design like actually choosing the the the kernel. So one way to do this is knowing what the actual uh dynamics are. uh but also then understanding based on which kernel you choose how does this uh uh affect dynamic stability and Duncan has an interesting paper on that uh for ste which I believe can also lift to time mixtures finally there's accelerating acceleration options like Newton methods preconditioning gradient flows to completely avoid uh computing uh kernels Yeah. So that's sort of a rough outline of ST mixture inference. Uh it's available in numpyro. So deep probabilistic programming language. Uh it separates model from uh inference. It has a DSL for probabilistic programs embedded in Python. So you can have arbitrary Python in between random sites. It uses non-standard interpretation. And most importantly like for this talk I guess is that Stein mixture is a blackbox. uh inference engine in the system. Um, of course, we need to be able to take

derivatives, hardware, acceleration, linearity, and all that is handed off to Jax. So, to just build the layer on top of Jax. Yeah, there's been a lot of people involved in this. So first of all the Pyrus devs that helped me build the the um the inference engine Martin and Koviaak and Eupong and Akmmed was who did the first version of it uh which then evolved into our um into our uh uh the current version of Steinvi which is the name of the inference engine and then my co-authors so my list who first formulated this problem when he was at PhD with Patrick Smith and then Thomas Hammer who's my PI um has worked with B for the last couple of years and Christoff Li who's an expert in Stein's method at University of Luxembourg the key references sum up so if the introduction into this approach to Bashian inference uh is the first one then there's the two Stein papers uh from 2006 16 and 19 and then our uh my paper here which is going to be at IC law this year. Yeah, this the inference group and uh yeah at KU and yeah yeah thanks. Awesome. Thank you. All right, I'll ask a few questions and then if anyone watching along wants to ask a question, I'd be happy to. So to kind of pull back and resummarize, what makes KL divergence optimization for particle transport plausible where distributional transport is not plausible? Uh so the the key issue here is computing the normalizing constant. You need you need to be able to renormalize uh uh reormalize the density. So like if you choose a very nice base of functions such as so they were using this orthogonal uh uh polomial basis then you you can do it but it restricts sort of how expressive you are able to do uh be in terms of what you're transporting a particle methods you so you it scales proportional with the number of particles so it's cheap to compute Great. So you you mentioned that the compute is quadratic in the particle number. So what are the computational complexity considerations for this method? And then h how does that even kind of qualitatively relate to the MCMC approach? like if that's giving a better posterior estimate of the real paths, what are the conditions where MCMC is just straightforwardly a better method to use versus exploring how many particles etc for this method? So uh so first in terms of MCMC I was like in a toy example you would probably just use MCMC but it becomes truly intractable if you sort of if it's the issue if especially if you're using HMC or something like this is that it's becomes it you need to work on all data points right so because you need to compute the grade like when you're doing a simplex integration you need to compute at every step you need to compute the gradient with respect to every data point and That's why MCMC just doesn't that doesn't scale to the type of data uh data sets that we're sort of interested in. um in terms of how is the complexity with um compared between Steinver descent and SMI the key uh sort of the expensive part is computing the kernel you need to compute the pair-wise

components this is why it's n squ and we have the same kernel structure uh for uh SVD and smi though the difference here is that we can what you can do uh with five particles uh with stein mixtures you might not at all be able to do on conventional hardware uh with the Stein version uh version of grid descent if your problem is high dimensional enough. Okay, so from the live chat so first quick question from Gary wrote what language are you using? So you mentioned the software package just could you redescribe or maybe even show the software package or what programming language is it in and what is that domain specific language look like? Okay. Uh yeah sure. Uh it's so the first of all the language is in Python uh and it's um maybe should I share something else? Yeah, sure. One second. See if I have something lying about we can look at and show you. All right. Uh, one second. Ah, here we go. [Music] Okay. Yep. So, can you see it? Yep. It's a little small. If you could zoom in, but we see it. Bending the smooth. Uh okay. Well, I have to do like this. Uh we can see it. All right. Good. So, this is a this is a a via soal uh uh autoenccoder, right? So, what you're doing is you're taking you have you have a you have a a guide which takes uh a picture something uh down to something uh some to some lowdimensional model. So we call this the uh the the encoder and then we have a model which takes our latent low dimensional and back to text. Okay. And so what I'm showing you here is uh is the the the decoder. So what takes our latent representation and moving us back into the original uh uh original space. So you can think of this as compression maybe uh if you like. Um okay. And so see this is what the random sites look like. So you have you have a random variable here. Uh we're going to be we're going to be using um a non-standard interpretation. So like have some have a set of uh operators which operate on these sample sites or handlers which operate on these sample sites and make them behave differently than otherwise just straight executing Python. So here I have a random sample site it's over angles it's distributed according to a sign by variant on myis distribution. So it's just a normal like think of it as a normal on a Taurus. uh it has some parameters and then uh I get to uh I have some observations. So this is like conditioning same as conditioning on these angles that I've observed. Now these uh parameters for my sign graises I have some location some concentration these come in this case from applying uh a neural network or bashian neural network it's a uh um and and that's just coded up in flax. So that's just straight up standard uh bashian uh sort of uh you know network and then we sort of we can we can uh make it into a bash network by just introducing a prior over its uh over its uh over the weight. So this is what's happening here. We're making a bash neural network. We're saying we have a uh a recurrent neural network. Uh it's a random site called decoder. So we can apply our handlers

uh to this site. Uh and then uh yeah this this just makes sure that uh it's the right the par like the right parameters are being updated when we're doing our uh updates. Okay. So and that then uh given some some latent remember they said uh they gave us my some representation which I can then map into the parameters my sign on my system. So this just give you an idea about this is what the like what modeling looks like. we introduce uh random sites and we can do sort of arbitrary things in Python in between our random sites to manipulate whatever uh uh samples we have. That's the sort of the the one side of it and then when you then want to do inference well you're so we're setting up our inference method. So right now I'm just testing thing out. So it's just mean field operational inference. I have the model which is what I just showed you. I have a corresponding guide. So this is my Q I was talking about. This is the thing encoding my uh my space into this latent representation. I'm going to decide uh on the type of optimizer using Adam and then I'm saying I'm going to use uh uh just a standard uh evidence lower bound here. um index or numpy where we need to be explicit about our the state of random keys. That's what's going on here. And then I'm running my inference. So now I'm conditioning uh my uh the parameters the uh the parameters of the guide on the observations that I have available which are stored in fine site and from there I just dump the object and I have a separate thing which does my diagnostics. So I can get my predictive distribution out. I can see what does the Ramosandon plot look like. uh if I want to look at uh histograms of uh parameters, I can get those out uh from my predictive. So that's sort of the the high scale view of a typical implementation of logic in awesome. Okay, next question in the chat. Bert Burkers wrote in the slide showing attraction and repulsion there are two particles with vectors pointing in many directions. How should one interpret these directions and their angles? Should I? Yes, this better. Perfect. Good. Okay. So the uh the blue uh the blue arrow will always point towards the closest uh mean. So it's we're looking at a gshian distribution here and it just in a mobile right. So the the particles are just the closest mean will always be the center of the crux. Um so that's what the blue arrows uh are uh are showing you. And then the the if you aligned up uh the red arrows there are the forces acting on the particles by each other. And so they will all in a two system uh particle system they will always be plus and minus at least if you choose gshian kernel of the same expression. So they will be of the same magnitude. You have three particles it becomes more complex. And then the black and I should have mentioned this that that's the net effect of the two. Okay, so it's looking at when you add up these two uh these two components, then you get the net effect of the black uh the black arrow. So that's when you're then taking a step, you're moving in the direction of the

black arrow and then like depending on how you choose your epsilon, you might not take the full length of this uh uh of the of the course be some epsilon size of it. And just to kind of clarify, what tunes the relative strength of the attractive and the repulsive force? Like at the end of all of this procedure, how do you ensure that you haven't over or underestimated the variance envelope? Okay. So this particle like this system of equations will when we are at convergence this will there will be no force acting on any of them right and so this this this equation will be zero. Got it? So the particles engage in these attractive repulsive physics-like interactions amongst observations delta-like kind of hidden Gaussian observations that can be variance reinflated and that particle transported particle distribution. It's almost like recapitulating what the MC would give you from samples because you do want to sample from a posterior distribution with a finite number of spikes. And instead of actually just doing that gradient calculation at the data point level as done in MC here, you're learning these transport functions with certain conditions that lets you move the distribution project down to particles and reinflate the particles back in your contribution to this mixture model. Yeah. We're we're moving the particles so that they get towards the posterior. So like the data is all our observations are hidden in in terms in in steambridge in descent at least in in in our score right okay I guess just taking one step back and then if anybody else wants to write any questions how did you come to this problem for your posttock like what led you through studies and everything to want to target this uh Yeah. So, yeah. So, actually it was uh it was uh it was uh Akmed who so we my original P my PhD was on sort of trying to find uh to determine threedimensional structure of proteins given um given the sequence or the pro uh protein uh 3D structure bro and and when I when I started um we had Alpha come out sort of two months after um which made the which and what that sort of that was declared as a solution to the problem which made it uh a a bit of an interesting uh situation to be in but uh but Akmat had started working on this um cuz he thought it mapped well onto sort of so one thing we want right is we want to be able to uh with our with the probabilistic deep probabilistic pro uh program is we want to automate uh automate inference and we sort of we know that there like this can't be done entirely but we want something that's very expressive and cheap to compute with and and it's version of gradient descent has had these two attractive properties it's very cheap and easy to understand uh and it's cheap to compute with. So we sort of we built a stress descent uh inference engine and then we were looking at well we have this issue uh and we wanted to apply it to proteins uh eventually and so what I was showing you before is well now I'm getting to that phase because it turned out to be a lot harder than we

first expected to actually like getting the inference engine running was not as much work and as then finding out is this thing correct? Uh, and that's that what we spent the next while on couple of years. There's always a funny postto story. Um, what does it look like to go from the open- source code that you have to adapting it to a problem? Like what's the kind of recipe that somebody would take from from um that's one question. And then a second question, however you want to do this, is where do you see this method in terms of the landscape of like generative AI, pure neural network based approaches, etc. Take the first one uh first. So what we would do there is we would go uh and okay so we would import our method instead of uh instead of uh instead of SVI and then we would replace it here. And now of course we need to choose a kernel. So the RBF kernel that that's it. Okay. Just just to confirm that move because it's largely built upon the JS Pyro approach. It's not so different than just flipping an import for a model that's already written within that way. Okay. So, how about how how does somebody even if this is sort of a a basic question for pyro users? Let's say somebody's looking at, you know, the temperature and the humidity in a room and making a kind of active inference model where they're going to run a heater or a cooler or some other kind of, you know, how do we go from just the observables and the unobservables of our target system of interest into representing it in a way that allows it to be inferred using this probabilistic approach. Yeah. So, so what you're describing sounds like the sort of the basian network or factor graph uh right and that's the model I was showing you is how I write that off. So I like whenever I have a random variable and in this case this is not the greatest of examples to look at but uh but I guess I have a set right so I have a random variable here which is my latent and then I'm saying that that there's an arrow between said and and uh and my my angles right and that's the same as saying like I have two uh I have a graph actually I can show you instead of telling Um let's see. So this is the sort of view that I think you were thinking of like we have random variables uh uh sort of here we have set and we have another one which is observed is angle but that's just that's just described as a python function which is model which is using these special uh specialized um uh like uh effectful uh methods one of them being sample and another one being what and a sample corresponds to to to a circle Here uh plate corresponds to what you would know in plate in a vation graph and then uh we have deterministics. So these are these uh dotted line circles. Awesome. And is this generated or specified in another file in this repo? How the bve model? Yeah. Yeah. That's so that's just the so the thing I was showing you at first this model is just that it's specifying this thing and it's just written as a function and that function I then act on with uh with my particular inference engine. Awesome. Okay. So

then kind of again pulling back like where do you see these methods in the toolkit of what people could do with with pure neural network approaches or transformer-based approaches today? So uh like so I mean so the Bashian neural network is sort of lifting this up right and like pretending that we can actually get access to a to the posterior of its parameters instead of point estimates and and there are I think I like the idea of vision neural networks because exactly because they sort of at least seems like they could give us an idea about risk or uncertainty. There's couple of problems with it, right? Like it's we have like moving you have you have two networks which look different in terms of weight but represent the same function. So like whether we are actually whether basian networks are actually like uh whether we when we're moving in weight space whether we're actually getting different sort of samples even though we're changing uh changing our weights is not clear and so so so it's not really you can't really determine whether you get different uh different uh if you running something like epmc you wouldn't actually know that your particular samples or different networks necessarily. So like estimating how well you've done or even determining convergence becomes very hard. Uh but it does handle sort of misspecification quite well right like if if you have something that's exceedingly flexible you probably have the right model in there. So that's why I think basin networks are attractive but we need to address this non identifiability eventually and I might be able to do that with kernels but we don't know yet. We have some ideas. Cool. Well, is there anything else you want to show or add or like how can people other than reading the paper and checking out the repo learn more? Uh yeah, I mean there's sorry this is where you get started. So pyro.ai take a look. uh our version is the nonpyro version but pyro is uh also very uh large and uh very fully fledged uh deep probabilistic programming language which I highly encourage you to take a look at. Cool. Anything else you want to add? Okay, awesome. Thank you again Ola for joining. Super interesting. I I hope we can continue to learn and that people in the institute community explore like pyro methods, RX and fur and we can start to see where these different methods complement each other, top differences, all that. Cool. Thank you. Bye. Bye.